Premier essaie de génération d'images

In [6]:
```
# Suppression des warnings FutureWarning
import warnings  # Gère les warnings
warnings.filterwarnings("ignore", category=FutureWarning)

# Librairies générales
import pandas as pd  # Manipulation de données tabulaires
import numpy as np  # Calcul scientifique et manipulation de matrices
import os  # Interaction avec le système de fichiers
from os.path import isfile, join  # Fonctions de gestion des chemins de fichiers
import sys  # Interaction avec l'interpréteur Python
import random  # Génération de nombres aléatoiresr
import zipfile  # Manipulation d'archives zip
from scipy.stats import norm, entropy # Statistiques sur les données

# Librairie d'affichage
import matplotlib.pyplot as plt  # Visualisation de données
from matplotlib.offsetbox import OffsetImage, AnnotationBbox  # Visualisation de d
from PIL import Image, ImageDraw  # Traitement d'images avec Pillow

# Bibliothèque scikit-learn
from sklearn.decomposition import PCA  # Réduction de dimension
from sklearn.manifold import TSNE  # Visualisation de données en dimension réduite

# TensorFlow et Keras
import tensorflow as tf  # API principale de TensorFlow
from tensorflow.keras.losses import mse  # Perte par erreur quadratique moyenne
from keras import layers, models, optimizers  # Création et gestion des modèles
from tensorflow.keras.preprocessing.image import ImageDataGenerator # Générateur
from keras.preprocessing.image import img_to_array, load_img, array_to_img  # Prép
from keras.callbacks import ModelCheckpoint, EarlyStopping  # Gestion des callback
from keras.layers import (Input, Conv2D, Flatten, Dense, Conv2DTranspose, Reshape,
                          Lambda, Activation, BatchNormalization, LeakyReLU, Drop
                          MaxPooling2D, UpSampling2D)  # Différentes couches pour
from keras.optimizers import Adam  # Optimiseur Adam
from keras.models import Model, load_model  # Définition et chargement des modèles
from keras.layers import LeakyReLU  # Activation LeakyReLU
import tensorflow.keras.backend as K # Import des opérations bas niveau de Keras
```

Définition des répertoires pour stocker les modéles appris

In [7]:
```
# Définition du répertoire cible
model_dir = "./Data_humanface_cat_without_caption_1000/"

# Création du répertoire s'il n'existe pas
os.makedirs(model_dir, exist_ok=True)

zip_file = "Data_humanface_cat_without_caption_1000.zip"

!wget https://www.lirmm.fr/~poncelet/Ressources/Data_humanface_cat_without_caption


# Extraction du fichier ZIP
with zipfile.ZipFile(zip_file, "r") as zip_ref:
        zip_ref.extractall(model_dir)

# Suppression du fichier ZIP après extraction pour économiser de l'espace
os.remove(zip_file)
```

```
--2026-05-03 13:42:38--  https://www.lirmm.fr/~poncelet/Ressources/Data_humanface_
cat_without_caption_1000.zip
Resolving www.lirmm.fr (www.lirmm.fr)... 193.49.104.251
Connecting to www.lirmm.fr (www.lirmm.fr)|193.49.104.251|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 42306490 (40M) [application/zip]
Saving to: ‘Data_humanface_cat_without_caption_1000.zip’

Data_humanface_cat_ 100%[===================>]  40.35M   972KB/s    in 55s

2026-05-03 13:43:31 (751 KB/s) - ‘Data_humanface_cat_without_caption_1000.zip’ sav
ed [42306490/42306490]
```

another way to download data idk which is better? # Nom local de l'archive et URL source zip_file = "Data_humanface_cat_without_caption_.zip" zip_url = "https://www.lirmm.fr/~poncelet/Ressources/Data_humanface_cat_without_caption_400.zip" # Répertoire cible après extraction src_dir = "images" image_dir = os.path.join(src_dir, "man") # Vérifie si le dossier de destination contient des images def humans_already_present(): return os.path.exists(image_dir) and any(f.endswith(".jpg") for f in os.listdir(image_dir)) # Téléchargement et extraction seulement si besoin if not humans_already_present(): print("Aucune image trouvée dans Cat. Téléchargement et extraction en cours...") # Télécharger si l'archive n'est pas déjà présente if not os.path.exists(zip_file): print(f"Téléchargement de {zip_file}...") !wget -O {zip_file} {zip_url} else: print(f"{zip_file} déjà présent, téléchargement ignoré.") # Extraction dans le répertoire courant with zipfile.ZipFile(zip_file, "r") as zip_ref: for member in zip_ref.namelist(): if not member.startswith("__MACOSX"): zip_ref.extract(member, ".") print("Archive extraite dans le répertoire courant.") # Suppression de l'archive après extraction os.remove(zip_file) else: print("Le dossier Humans/train existe déjà et contient des images. Téléchargement ignoré.")

In [8]:
```
##affichage des images
```

In [9]:
```
# Répertoire contenant les images
src_dir = "Data_humanface_cat_without_caption_1000/images"
image_dir = os.path.join(src_dir, "cat")

files = [f for f in os.listdir(image_dir) if f.endswith(".jpg")]

print("Nombre d'images dans", image_dir, ":", len(files))

if files:
    with Image.open(os.path.join(image_dir, files[0])) as img:
        shape = img.size[::-1] + (3,)  # (height, width, 3)
        print("Format (shape) des images :", shape)


# Tirage de 5 images aléatoires
random_files = random.sample(files, 5)

# Affichage
plt.figure(figsize=(8, 2))
for i, fname in enumerate(random_files):
    img_path = os.path.join(image_dir, fname)
    img = Image.open(img_path)

    plt.subplot(1, 5, i + 1)
    plt.imshow(img)
    plt.title(fname, fontsize=8)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Nombre d'images dans Data_humanface_cat_without_caption_1000/images/cat : 1000
Format (shape) des images : (256, 256, 3)
```

2. Importer le jeu de données

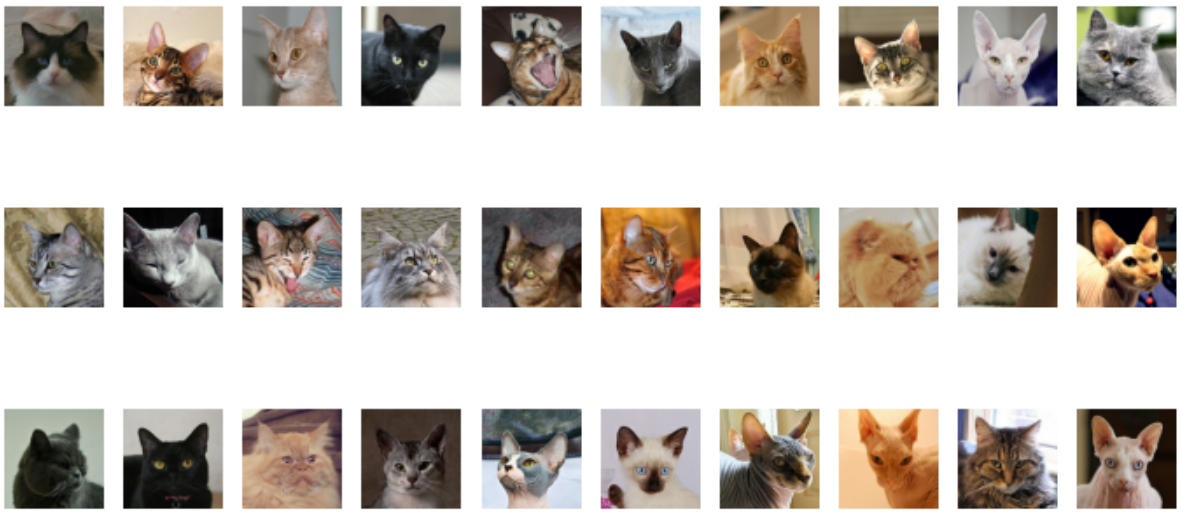
In [10]:

```
image_size = (128, 128, 3)  # Dimensions des images (hauteur, largeur, canaux)
batch_size = 512             # Taille des lots
latent_dim = 200             # Dimension du vecteur latent pour l'autoencodeur

# Génération des données normalisées (valeurs entre 0 et 1)
data_flow = ImageDataGenerator(rescale=1./255).flow_from_directory(
    directory=src_dir,
    classes=["cat"],  # Le sous-dossier contenant toutes les images
    target_size=image_size[:2],  # Redimension à 128x128
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input'  # Autoencodeur : entrée = sortie
)
```

```
Found 1000 images belonging to 1 classes.
```

In [11]:

```
# Obtention du premier batch d'images générées
data_batch = next(data_flow)

# Affichage des 30 premières images du batch
import matplotlib.pyplot as plt

fig, axes = plt.subplots(3, 10, figsize=(12, 6))
for i, ax in enumerate(axes.flatten()):
    ax.imshow(data_batch[0][i])
    ax.axis('off')
plt.show()
```



1/ Définition de l'encodeur et décodeur

In [12]:

```
def build_encoder(input_dim, output_dim):
    tf.keras.backend.clear_session()
```

```
    encoder_input = Input(shape=input_dim, name='encoder_input')
    x = encoder_input

    x = Conv2D(32, kernel_size=3, strides=2, padding='same',
               name='encoder_conv_1')(x)
    x = LeakyReLU(name='encoder_act_1')(x)

    x = Conv2D(64, kernel_size=3, strides=2, padding='same',
               name='encoder_conv_2')(x)
    x = LeakyReLU(name='encoder_act_2')(x)

    x = Conv2D(64, kernel_size=3, strides=2, padding='same',
               name='encoder_conv_3')(x)
    x = LeakyReLU(name='encoder_act_3')(x)

    x = Conv2D(64, kernel_size=3, strides=2, padding='same',
               name='encoder_conv_4')(x)
    x = LeakyReLU(name='encoder_act_4')(x)

    shape_before_flattening = tf.keras.backend.int_shape(x)[1:]
    x = Flatten()(x)
    encoder_output = Dense(output_dim, name='encoder_output')(x)

    encoder = Model(encoder_input, encoder_output, name='encoder')
    return encoder_input, encoder_output, shape_before_flattening, encoder

def build_decoder(input_dim, shape_before_flattening):
    # Entrée du décodeur (vecteur latent)
    decoder_input = Input(shape=(input_dim,), name='decoder_input')

    # Projeter dans un tenseur de même forme qu'avant le Flatten de l'encodeur
    x = Dense(np.prod(shape_before_flattening))(decoder_input)
    x = Reshape(shape_before_flattening)(x)

    # Couches de déconvolution (effet miroir de l'encodeur)
    x = Conv2DTranspose(64, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_1')(x)
    x = LeakyReLU(name='decoder_act_1')(x)

    x = Conv2DTranspose(64, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_2')(x)
    x = LeakyReLU(name='decoder_act_2')(x)

    x = Conv2DTranspose(32, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_3')(x)
    x = LeakyReLU(name='decoder_act_3')(x)

    x = Conv2DTranspose(3, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_4')(x)
    decoder_output = Activation('sigmoid', name='decoder_output')(x)

    decoder = Model(decoder_input, decoder_output, name='decoder')
    return decoder_input, decoder_output, decoder
```

In [13]:

```
image_size = (128, 128, 3)  # Dimensions des images (hauteur, largeur, canaux)
batch_size = 512            # Taille des lots
latent_dim = 200            # Dimension du vecteur latent pour l'autoencodeur

# Création de l'encodeur avec les dimensions d'entrée et de sortie spécifiées
encoder_input, encoder_output, shape_before_flattening, encoder = build_encoder(
    input_dim=image_size,
    output_dim=latent_dim
```

```
)

# Création du décodeur en utilisant la dimension de l'espace latent
# et le shape  avant le flattening
decoder_input, decoder_output, decoder = build_decoder(
    input_dim=latent_dim,
    shape_before_flattening=shape_before_flattening
)
```

```
2026-05-03 13:43:46.610610: E external/local_xla/xla/stream_executor/cuda/cuda_pla
tform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: U
NKNOWN ERROR (303)
```

Partie autoencodeur

In [14]:

```
# L'entrée du modèle autoencodeur est celle de l'encodeur
autoencoder_input = encoder_input

# La sortie est obtenue en appliquant le décodeur à la sortie de l'encodeur
autoencoder_output = decoder(encoder_output)

# Création du modèle autoencodeur complet (encodeur + décodeur)
autoencoder = Model(autoencoder_input, autoencoder_output, name='autoencoder')

# Affichage de la structure du modèle
autoencoder.summary()
```

Model: "autoencoder"

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	(None, 128, 128, 3)	0
encoder_conv_1 (Conv2D)	(None, 64, 64, 32)	896
encoder_act_1 (LeakyReLU)	(None, 64, 64, 32)	0
encoder_conv_2 (Conv2D)	(None, 32, 32, 64)	18,496
encoder_act_2 (LeakyReLU)	(None, 32, 32, 64)	0
encoder_conv_3 (Conv2D)	(None, 16, 16, 64)	36,928
encoder_act_3 (LeakyReLU)	(None, 16, 16, 64)	0
encoder_conv_4 (Conv2D)	(None, 8, 8, 64)	36,928
encoder_act_4 (LeakyReLU)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
encoder_output (Dense)	(None, 200)	819,400
decoder (Functional)	(None, 128, 128, 3)	916,483

Total params: 1,829,131 (6.98 MB)

Trainable params: 1,829,131 (6.98 MB)

Non-trainable params: 0 (0.00 B)

In [15]:

```
# Génération des données normalisées (valeurs entre 0 et 1)
data_flow = ImageDataGenerator(rescale=1./255).flow_from_directory(
    directory=src_dir,
    classes=['cat'],  # Le sous-dossier contenant toutes les images
    target_size=image_size[:2],  # Redimension à 128x128
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input'  # Autoencodeur : entrée = sortie
)
```

```
Found 1000 images belonging to 1 classes.
```

Espace latent

In [16]:

```
def visualize_latent_space(encoder_model, images, num_images=100,
                           image_size=(28, 28), zoom=0.6):
    """
    Visualise la répartition de l'espace latent en utilisant t-SNE
    et affiche des vignettes d'images.

    Paramètres :
    -----------
    encoder_model : Model
        Le modèle d'encodeur (autoencodeur ou VAE) à utiliser pour obtenir
        les vecteurs latents.
    images : array-like
        Batch d'images d'entrée pour la visualisation.
    num_images : int, optional
        Nombre d'images à utiliser pour la visualisation (par défaut 100).
    image_size : tuple, optional
        Taille des vignettes d'image à afficher (par défaut (28, 28)).
    zoom : float, optional
        Facteur de zoom pour les vignettes d'image (par défaut 0.6).
    """
    # Limite le nombre d'images à `num_images`
    images = images[:num_images]

    # Génère les vecteurs latents avec l'encodeur
    latent_vectors = encoder_model.predict(images)
    if isinstance(latent_vectors, list):
        latent_vectors = latent_vectors[0]

    # Aplatit les vecteurs latents s'ils ont plus de 2 dimensions
    if latent_vectors.ndim > 2:
        latent_vectors = latent_vectors.reshape((latent_vectors.shape[0], -1))

    # Réduction de dimension avec t-SNE pour projeter en 2D
    tsne = TSNE(n_components=2, random_state=42)
    latent_2d = tsne.fit_transform(latent_vectors)

    plt.figure(figsize=(15, 12))
    ax = plt.gca()

    # Affichage des images dans l'espace t-SNE avec AnnotationBbox
    for i, (x, y) in enumerate(latent_2d):
        # Conversion de l'image en vignette
        thumbnail = array_to_img(images[i])
        thumbnail = thumbnail.resize(image_size)
        imagebox = OffsetImage(thumbnail, zoom=zoom)
        ab = AnnotationBbox(imagebox, (x, y), frameon=False)
        ax.add_artist(ab)
```

```
    all_x, all_y = latent_2d[:, 0], latent_2d[:, 1]
    ax.set_xlim(all_x.min() - 5, all_x.max() + 5)
    ax.set_ylim(all_y.min() - 5, all_y.max() + 5)

    plt.title("Répartition des images dans l'espace latent")
    plt.xlabel("Dimension 1")
    plt.ylabel("Dimension 2")

    ax.grid(False)
    plt.show()
```

In [17]:

```
#my guess is we need this before?
# Définition des hyperparamètres
epochs = 150
model_name = "catmodeltrain.keras"
model_path = os.path.join(model_dir, model_name)

# Compilation du modèle autoencodeur
autoencoder.compile(
    optimizer='adam',
    loss='binary_crossentropy'
)

# Générateur principal avec normalisation  et séparation train/validation
train_gen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

# Générateur pour les données d'entraînement
train_flow = train_gen.flow_from_directory(
    directory=src_dir,
    classes=['cat'],  # le sous-dossier contenant toutes les images
    target_size=image_size[:2],
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input',
    subset='training'
)

# Générateur pour les données de validation
val_flow = train_gen.flow_from_directory(
    directory=src_dir,
    classes=['cat'],
    target_size=image_size[:2],
    batch_size=batch_size,
    shuffle=False,
    seed=42,
    class_mode='input',
    subset='validation'
)


# Entraînement du modèle autoencodeur
#autoencoder = train_autoencoder_from_generator(
   # model=autoencoder,
    #train_gen=train_flow,
    #val_gen=val_flow,
    #model_path=model_path,
    #epochs=epochs,
```

```
    #patience=5
#)
```

```
Found 800 images belonging to 1 classes.
Found 200 images belonging to 1 classes.
```

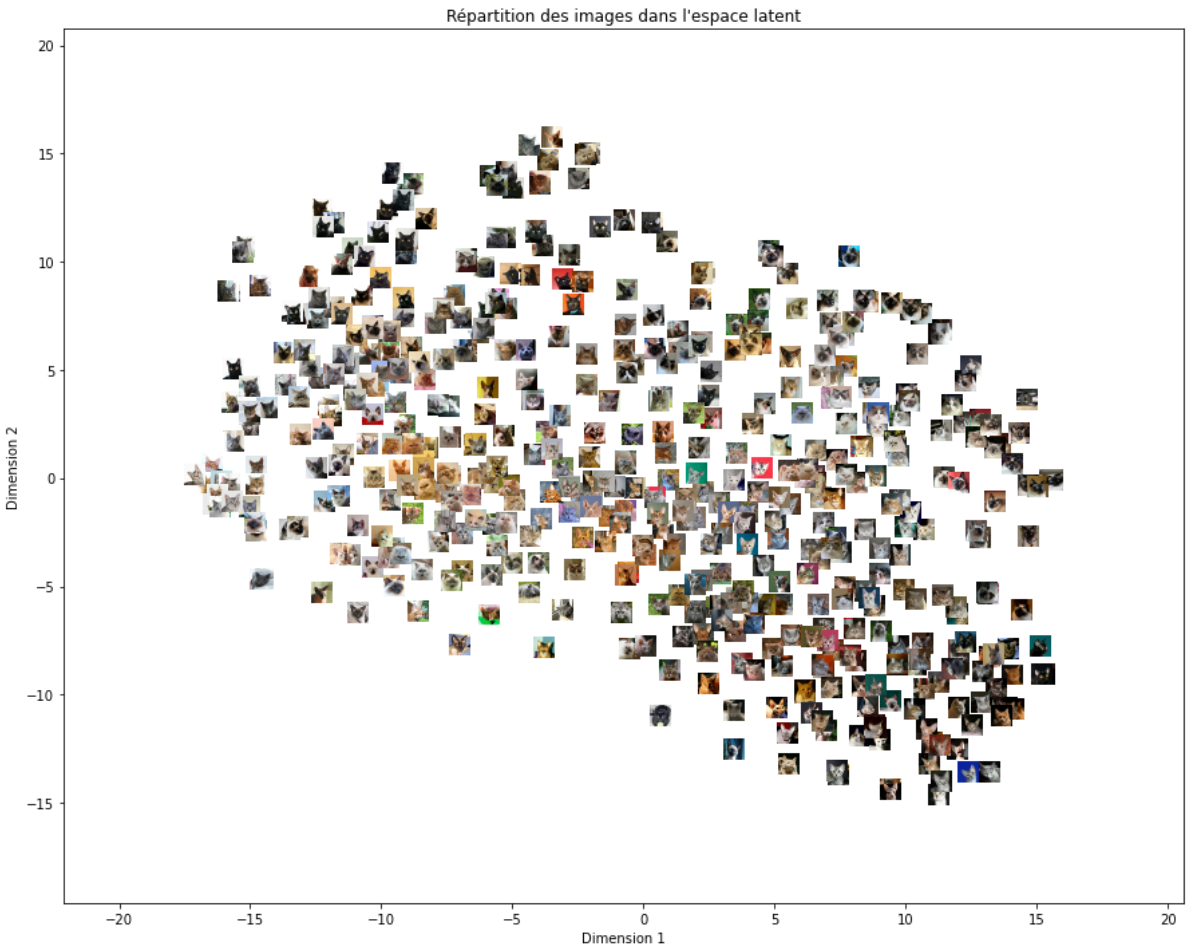
In [18]:

```
# Chemin vers le modèle entraîné
model_path = os.path.join(model_dir, "catmodeltrain.keras")

# Chargement du modèle
autoencoder = load_model(model_path)

# Récupération du premier lot d'images
example_batch = next(data_flow)

# Extraction des images
example_batch = example_batch[0]
```

In [19]:

```
# Visualisation de l'espace latent avec les vignettes
visualize_latent_space(
    encoder_model=encoder,
    images=example_batch,
    num_images=batch_size,
    image_size=(32, 32),
    zoom=0.5
)
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step
```



Transformation auto-encodeur en VAEOn modifie l'encodeur : On veut un encodeur qui à chaque image d'entrée retourne une distribution dans l'espace latent avec 2 paramétres: moyenne et log variance 1. on inclut la couche de perte

In [23]:

```
image_size = (128, 128, 3)  # Dimensions des images (hauteur, largeur, canaux)
batch_size = 512             # Taille des lots
latent_dim = 200             # Dimension du vecteur latent pour l'autoencodeur
```

In [24]:

```
import tensorflow as tf
# Fonction de sampling avec reparameterization trick
@tf.keras.utils.register_keras_serializable()
def sampling(args):
    mean_mu, log_var = args
    epsilon = K.random_normal(shape=K.shape(mean_mu), mean=0., stddev=1.)
    return mean_mu + K.exp(log_var / 2) * epsilon

# Couche personnalisée pour le calcul de la perte KL
@tf.keras.utils.register_keras_serializable()
class KLLossLayer(tf.keras.layers.Layer):
    def call(self, inputs):
        mean_mu, log_var = inputs
        kl_loss = -0.5 * tf.reduce_sum(1 + log_var - tf.square(mean_mu) - tf.exp(l
        self.add_loss(kl_loss)
        return kl_loss

# VAE Encoder
def build_vae_encoder(input_dim, output_dim):
    tf.keras.backend.clear_session()

    vae_encoder_input = Input(shape=input_dim, name='vae_encoder_input')
    x = vae_encoder_input

    x = Conv2D(32, 3, strides=2, padding='same', name='vae_encoder_conv_1')(x)
    x = LeakyReLU(name='vae_encoder_act_1')(x)

    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_2')(x)
    x = LeakyReLU(name='vae_encoder_act_2')(x)

    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_3')(x)
    x = LeakyReLU(name='vae_encoder_act_3')(x)

    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_4')(x)
    x = LeakyReLU(name='vae_encoder_act_4')(x)

    shape_before_flattening = tf.keras.backend.int_shape(x)[1:]
    x = Flatten(name='vae_encoder_flatten')(x)

    mean_mu = Dense(output_dim, name='mu')(x)
    log_var = Dense(output_dim, name='log_var')(x)

    vae_encoder_output = Lambda(sampling, output_shape=(output_dim,),
                                name='vae_encoder_output')([mean_mu, log_var])

    _ = KLLossLayer(name='kl_loss')([mean_mu, log_var])

    vae_encoder = Model(vae_encoder_input, vae_encoder_output, name='vae_encoder')
    return vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_f
```

In [25]:

```
# Construction de l'encodeur VAE
vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_flattening,
    input_dim=image_size,
    output_dim=latent_dim
)
```

```
# Affichage de la structure de l'encodeur
vae_encoder.summary()
```

Model: "vae_encoder"

Layer (type)	Output Shape	Param #	Connected to
vae_encoder_input (InputLayer)	(None, 128, 128, 3)	0	-
vae_encoder_conv_1 (Conv2D)	(None, 64, 64, 32)	896	vae_encoder_inpu…
vae_encoder_act_1 (LeakyReLU)	(None, 64, 64, 32)	0	vae_encoder_conv…
vae_encoder_conv_2 (Conv2D)	(None, 32, 32, 64)	18,496	vae_encoder_act_…
vae_encoder_act_2 (LeakyReLU)	(None, 32, 32, 64)	0	vae_encoder_conv…
vae_encoder_conv_3 (Conv2D)	(None, 16, 16, 64)	36,928	vae_encoder_act_…
vae_encoder_act_3 (LeakyReLU)	(None, 16, 16, 64)	0	vae_encoder_conv…
vae_encoder_conv_4 (Conv2D)	(None, 8, 8, 64)	36,928	vae_encoder_act_…
vae_encoder_act_4 (LeakyReLU)	(None, 8, 8, 64)	0	vae_encoder_conv…
vae_encoder_flatten (Flatten)	(None, 4096)	0	vae_encoder_act_…
mu (Dense)	(None, 200)	819,400	vae_encoder_flat…
log_var (Dense)	(None, 200)	819,400	vae_encoder_flat…
vae_encoder_output (Lambda)	(None, 200)	0	mu[0][0], log_var[0][0]

Total params: 1,732,048 (6.61 MB)

Trainable params: 1,732,048 (6.61 MB)

Non-trainable params: 0 (0.00 B)

même décodeur donc pas besoin de redefinir

In [26]:

```
# Construction du décodeur VAE
decoder_input, decoder_output, decoder = build_decoder(
    input_dim=latent_dim,
    shape_before_flattening=shape_before_flattening
)

# Affichage de la structure du décodeur
decoder.summary()
```

Model: "decoder"

Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	(None, 200)	0
dense (Dense)	(None, 4096)	823,296
reshape (Reshape)	(None, 8, 8, 64)	0
decoder_conv_1 (Conv2DTranspose)	(None, 16, 16, 64)	36,928
decoder_act_1 (LeakyReLU)	(None, 16, 16, 64)	0
decoder_conv_2 (Conv2DTranspose)	(None, 32, 32, 64)	36,928
decoder_act_2 (LeakyReLU)	(None, 32, 32, 64)	0
decoder_conv_3 (Conv2DTranspose)	(None, 64, 64, 32)	18,464
decoder_act_3 (LeakyReLU)	(None, 64, 64, 32)	0
decoder_conv_4 (Conv2DTranspose)	(None, 128, 128, 3)	867
decoder_output (Activation)	(None, 128, 128, 3)	0

Total params: 916,483 (3.50 MB)

Trainable params: 916,483 (3.50 MB)

Non-trainable params: 0 (0.00 B)

En combinant l'encodeur variationnel et le décodeur pour former notre VAE :

In [27]:

```
# Initialisation de l'encodeur VAE
vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_flattening,
    input_dim=image_size,
    output_dim=latent_dim
)

# Initialisation du décodeur (identique à l'autoencodeur classique)
decoder_input, decoder_output, decoder = build_decoder(
    input_dim=latent_dim,
    shape_before_flattening=shape_before_flattening
)

# Construction du VAE complet
vae_output = decoder(vae_encoder_output)
vae = Model(inputs=vae_encoder_input, outputs=vae_output, name="VAE")

# Compilation du modèle avec la reconstruction loss (la KL est intégrée
# dans l'encodeur)
B = 1.0  # facteur de pondération de la reconstruction

def r_loss(y_true, y_pred):
    return K.mean(K.square(y_true - y_pred), axis=[1, 2, 3]) # MSE pixel à pixel
```

```python
def total_loss(y_true, y_pred):
    return B * r_loss(y_true, y_pred) # KL-divergence ajoutée via .add_loss()

vae.compile(optimizer=Adam(), loss=total_loss)

# Affichage du résumé du modèle
vae.summary()
```

Model: "VAE"

Layer (type)	Output Shape	Param #	Connected to
vae_encoder_input (InputLayer)	(None, 128, 128, 3)	0	-
vae_encoder_conv_1 (Conv2D)	(None, 64, 64, 32)	896	vae_encoder_inpu…
vae_encoder_act_1 (LeakyReLU)	(None, 64, 64, 32)	0	vae_encoder_conv…
vae_encoder_conv_2 (Conv2D)	(None, 32, 32, 64)	18,496	vae_encoder_act_…
vae_encoder_act_2 (LeakyReLU)	(None, 32, 32, 64)	0	vae_encoder_conv…
vae_encoder_conv_3 (Conv2D)	(None, 16, 16, 64)	36,928	vae_encoder_act_…
vae_encoder_act_3 (LeakyReLU)	(None, 16, 16, 64)	0	vae_encoder_conv…
vae_encoder_conv_4 (Conv2D)	(None, 8, 8, 64)	36,928	vae_encoder_act_…
vae_encoder_act_4 (LeakyReLU)	(None, 8, 8, 64)	0	vae_encoder_conv…
vae_encoder_flatten (Flatten)	(None, 4096)	0	vae_encoder_act_…
mu (Dense)	(None, 200)	819,400	vae_encoder_flat…
log_var (Dense)	(None, 200)	819,400	vae_encoder_flat…
vae_encoder_output (Lambda)	(None, 200)	0	mu[0][0], log_var[0][0]
decoder (Functional)	(None, 128, 128, 3)	916,483	vae_encoder_outp…

Total params: 2,648,531 (10.10 MB)

Trainable params: 2,648,531 (10.10 MB)

Non-trainable params: 0 (0.00 B)

définition de la fonction train adapté au VAE

In [29]:

```
# Génération des données normalisées (valeurs entre 0 et 1)
data_flow = ImageDataGenerator(rescale=1./255).flow_from_directory(
    directory=src_dir,
    classes=['cat'],  # Le sous-dossier contenant toutes les images
    target_size=image_size[:2],  # Redimension à 128x128
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input'  # Autoencodeur : entrée = sortie
)
```

```
Found 1000 images belonging to 1 classes.
```

In [30]:

```
def train_vae_from_generator(model, train_gen, val_gen,
                             model_path, epochs=100, patience=5):
    """
    Entraîne un modèle VAE avec des générateurs Keras.

    Args:
        model       : modèle VAE compilé
        train_gen   : générateur pour l'entraînement
        val_gen     : générateur pour la validation
        model_path  : chemin de sauvegarde du meilleur modèle
        epochs      : nombre maximal d'époques
        patience    : patience pour l'arrêt anticipé

    Returns:
        Le modèle avec les meilleurs poids restaurés.
    """
    callbacks = [
        EarlyStopping(monitor='val_loss', patience=patience,
                      restore_best_weights=True, verbose=1),
        ModelCheckpoint(filepath=model_path,
                        monitor='val_loss',
                        save_best_only=True,
                        save_weights_only=False,
                        verbose=1)
    ]

    history = model.fit(
        train_gen,
        validation_data=val_gen,
        epochs=epochs,
        callbacks=callbacks,
        verbose=1
    )

    return model
```

In [31]:

```
# Définition des hyperparamètres
epochs = 150
model_name = "catGeneratingVAE.keras"
model_path = os.path.join(model_dir, model_name)

# Générateur principal avec normalisation et séparation train/validation
train_gen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

# Générateur pour les données d'entraînement
```

```
train_flow = train_gen.flow_from_directory(
    directory=src_dir,
    classes=['cat'],  # le sous-dossier contenant toutes les images
    target_size=image_size[:2],
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input',
    subset='training'
)

# Générateur pour les données de validation
val_flow = train_gen.flow_from_directory(
    directory=src_dir,
    classes=['cat'],
    target_size=image_size[:2],
    batch_size=batch_size,
    shuffle=False,
    seed=42,
    class_mode='input',
    subset='validation'
)

# Entraînement du modèle VAE
vae = train_vae_from_generator(
    model=vae,
    train_gen=train_flow,
    val_gen=val_flow,
    model_path=model_path,
    epochs=epochs,
    patience=5
)
```

```
Found 800 images belonging to 1 classes.
Found 200 images belonging to 1 classes.
Epoch 1/150
WARNING: All log messages before absl::InitializeLog() is called are written to ST
DERR
E0000 00:00:1777809044.222772    1508 meta_optimizer.cc:967] remapper failed: INVA
LID_ARGUMENT: Mutation::Apply error: fanout 'StatefulPartitionedCall/gradient_tap
e/VAE_1/vae_encoder_act_2_1/LeakyRelu/LeakyReluGrad' exist for missing node 'State
fulPartitionedCall/VAE_1/vae_encoder_conv_2_1/BiasAdd'.
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0729
Epoch 1: val_loss improved from None to 0.07462, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 7s 4s/step - loss: 0.0733 - val_loss: 0.0746
Epoch 2/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0737
Epoch 2: val_loss improved from 0.07462 to 0.07446, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0731 - val_loss: 0.0745
Epoch 3/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 809ms/step - loss: 0.0724
Epoch 3: val_loss improved from 0.07446 to 0.07428, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0729 - val_loss: 0.0743
Epoch 4/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0733
Epoch 4: val_loss improved from 0.07428 to 0.07406, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0727 - val_loss: 0.0741
Epoch 5/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0722
Epoch 5: val_loss improved from 0.07406 to 0.07377, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0723 - val_loss: 0.0738
Epoch 6/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 927ms/step - loss: 0.0712
Epoch 6: val_loss improved from 0.07377 to 0.07343, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0720 - val_loss: 0.0734
Epoch 7/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 931ms/step - loss: 0.0712
Epoch 7: val_loss improved from 0.07343 to 0.07305, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0715 - val_loss: 0.0730
Epoch 8/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0719
Epoch 8: val_loss improved from 0.07305 to 0.07277, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 629ms/step - loss: 0.0709 - val_loss: 0.0728
Epoch 9/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - loss: 0.0714
Epoch 9: val_loss improved from 0.07277 to 0.07272, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0704 - val_loss: 0.0727
Epoch 10/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0685
Epoch 10: val_loss improved from 0.07272 to 0.07258, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0701 - val_loss: 0.0726
Epoch 11/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0692
Epoch 11: val_loss improved from 0.07258 to 0.07175, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0698 - val_loss: 0.0718
Epoch 12/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 940ms/step - loss: 0.0700
Epoch 12: val_loss improved from 0.07175 to 0.07008, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0690 - val_loss: 0.0701
Epoch 13/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 875ms/step - loss: 0.0682
Epoch 13: val_loss improved from 0.07008 to 0.06638, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0675 - val_loss: 0.0664
```

```
Epoch 14/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 901ms/step - loss: 0.0634
Epoch 14: val_loss improved from 0.06638 to 0.06180, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0629 - val_loss: 0.0618
Epoch 15/150
1/2 ━━━━━━━━━━──────────  3s 4s/step - loss: 0.0597
Epoch 15: val_loss improved from 0.06180 to 0.05650, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s -135670us/step - loss: 0.0594 - val_loss: 0.0565
Epoch 16/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0543
Epoch 16: val_loss improved from 0.05650 to 0.05427, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0541 - val_loss: 0.0543
Epoch 17/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0538
Epoch 17: val_loss improved from 0.05427 to 0.05111, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0536 - val_loss: 0.0511
Epoch 18/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0490
Epoch 18: val_loss improved from 0.05111 to 0.04935, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0499 - val_loss: 0.0493
Epoch 19/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0473
Epoch 19: val_loss improved from 0.04935 to 0.04666, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0470 - val_loss: 0.0467
Epoch 20/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0455
Epoch 20: val_loss did not improve from 0.04666
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0454 - val_loss: 0.0482
Epoch 21/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 950ms/step - loss: 0.0449
Epoch 21: val_loss improved from 0.04666 to 0.04306, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0443 - val_loss: 0.0431
Epoch 22/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 936ms/step - loss: 0.0423
Epoch 22: val_loss improved from 0.04306 to 0.04208, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0415 - val_loss: 0.0421
Epoch 23/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0400
Epoch 23: val_loss improved from 0.04208 to 0.03793, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0394 - val_loss: 0.0379
Epoch 24/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0362
Epoch 24: val_loss improved from 0.03793 to 0.03548, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0362 - val_loss: 0.0355
Epoch 25/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0344
Epoch 25: val_loss improved from 0.03548 to 0.03400, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0343 - val_loss: 0.0340
Epoch 26/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 859ms/step - loss: 0.0330
Epoch 26: val_loss improved from 0.03400 to 0.03283, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0328 - val_loss: 0.0328
```

```
Epoch 27/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 939ms/step - loss: 0.0316
Epoch 27: val_loss improved from 0.03283 to 0.03133, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0316 - val_loss: 0.0313
Epoch 28/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0304
Epoch 28: val_loss improved from 0.03133 to 0.03028, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0302 - val_loss: 0.0303
Epoch 29/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0294
Epoch 29: val_loss improved from 0.03028 to 0.02946, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0293 - val_loss: 0.0295
Epoch 30/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0285
Epoch 30: val_loss improved from 0.02946 to 0.02808, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0282 - val_loss: 0.0281
Epoch 31/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 843ms/step - loss: 0.0272
Epoch 31: val_loss improved from 0.02808 to 0.02728, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0271 - val_loss: 0.0273
Epoch 32/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0264
Epoch 32: val_loss improved from 0.02728 to 0.02674, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0262 - val_loss: 0.0267
Epoch 33/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 844ms/step - loss: 0.0255
Epoch 33: val_loss did not improve from 0.02674
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0257 - val_loss: 0.0270
Epoch 34/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0263
Epoch 34: val_loss did not improve from 0.02674
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0265 - val_loss: 0.0285
Epoch 35/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0262
Epoch 35: val_loss did not improve from 0.02674
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0258 - val_loss: 0.0276
Epoch 36/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0258
Epoch 36: val_loss improved from 0.02674 to 0.02661, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0252 - val_loss: 0.0266
Epoch 37/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0248
Epoch 37: val_loss improved from 0.02661 to 0.02450, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0247 - val_loss: 0.0245
Epoch 38/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 998ms/step - loss: 0.0238
Epoch 38: val_loss improved from 0.02450 to 0.02376, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0239 - val_loss: 0.0238
Epoch 39/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0234
Epoch 39: val_loss improved from 0.02376 to 0.02357, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0235 - val_loss: 0.0236
Epoch 40/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0231
```

```
Epoch 40: val_loss improved from 0.02357 to 0.02309, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0228 - val_loss: 0.0231
Epoch 41/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0221
Epoch 41: val_loss improved from 0.02309 to 0.02298, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0224 - val_loss: 0.0230
Epoch 42/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0221
Epoch 42: val_loss improved from 0.02298 to 0.02252, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0220 - val_loss: 0.0225
Epoch 43/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0215
Epoch 43: val_loss improved from 0.02252 to 0.02241, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0216 - val_loss: 0.0224
Epoch 44/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0211
Epoch 44: val_loss improved from 0.02241 to 0.02195, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0212 - val_loss: 0.0219
Epoch 45/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - loss: 0.0211
Epoch 45: val_loss improved from 0.02195 to 0.02190, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0208 - val_loss: 0.0219
Epoch 46/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0209
Epoch 46: val_loss improved from 0.02190 to 0.02139, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0206 - val_loss: 0.0214
Epoch 47/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0204
Epoch 47: val_loss improved from 0.02139 to 0.02111, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0203 - val_loss: 0.0211
Epoch 48/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 881ms/step - loss: 0.0202
Epoch 48: val_loss improved from 0.02111 to 0.02073, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0200 - val_loss: 0.0207
Epoch 49/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0194
Epoch 49: val_loss improved from 0.02073 to 0.02056, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0199 - val_loss: 0.0206
Epoch 50/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0193
Epoch 50: val_loss improved from 0.02056 to 0.02029, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0195 - val_loss: 0.0203
Epoch 51/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0194
Epoch 51: val_loss improved from 0.02029 to 0.02005, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0192 - val_loss: 0.0201
Epoch 52/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 943ms/step - loss: 0.0191
Epoch 52: val_loss improved from 0.02005 to 0.01984, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0190 - val_loss: 0.0198
Epoch 53/150
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0191
Epoch 53: val_loss improved from 0.01984 to 0.01984, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0188 - val_loss: 0.0198
Epoch 54/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 862ms/step - loss: 0.0186
Epoch 54: val_loss improved from 0.01984 to 0.01957, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0187 - val_loss: 0.0196
Epoch 55/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0182
Epoch 55: val_loss improved from 0.01957 to 0.01923, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0184 - val_loss: 0.0192
Epoch 56/150
1/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0181
Epoch 56: val_loss improved from 0.01923 to 0.01910, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s -407008us/step - loss: 0.0181 - val_loss: 0.0191
Epoch 57/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0181
Epoch 57: val_loss did not improve from 0.01910
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0180 - val_loss: 0.0191
Epoch 58/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0184
Epoch 58: val_loss did not improve from 0.01910
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0184 - val_loss: 0.0209
Epoch 59/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0201
Epoch 59: val_loss improved from 0.01910 to 0.01906, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0203 - val_loss: 0.0191
Epoch 60/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0184
Epoch 60: val_loss did not improve from 0.01906
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0188 - val_loss: 0.0206
Epoch 61/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0190
Epoch 61: val_loss did not improve from 0.01906
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0189 - val_loss: 0.0199
Epoch 62/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 914ms/step - loss: 0.0184
Epoch 62: val_loss did not improve from 0.01906
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0184 - val_loss: 0.0197
Epoch 63/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0180
Epoch 63: val_loss did not improve from 0.01906
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0177 - val_loss: 0.0193
Epoch 64/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0177
Epoch 64: val_loss improved from 0.01906 to 0.01889, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0176 - val_loss: 0.0189
Epoch 65/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0174
Epoch 65: val_loss improved from 0.01889 to 0.01866, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0173 - val_loss: 0.0187
Epoch 66/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0173
Epoch 66: val_loss improved from 0.01866 to 0.01849, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0171 - val_loss: 0.0185
Epoch 67/150
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0168
Epoch 67: val_loss improved from 0.01849 to 0.01821, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0169 - val_loss: 0.0182
Epoch 68/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 847ms/step - loss: 0.0170
Epoch 68: val_loss improved from 0.01821 to 0.01806, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0169 - val_loss: 0.0181
Epoch 69/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 908ms/step - loss: 0.0167
Epoch 69: val_loss improved from 0.01806 to 0.01784, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0168 - val_loss: 0.0178
Epoch 70/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0166
Epoch 70: val_loss improved from 0.01784 to 0.01755, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0167 - val_loss: 0.0176
Epoch 71/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 948ms/step - loss: 0.0165
Epoch 71: val_loss improved from 0.01755 to 0.01729, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s -38960us/step - loss: 0.0165 - val_loss: 0.0173
Epoch 72/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0165
Epoch 72: val_loss improved from 0.01729 to 0.01717, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0163 - val_loss: 0.0172
Epoch 73/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 847ms/step - loss: 0.0159
Epoch 73: val_loss improved from 0.01717 to 0.01714, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0161 - val_loss: 0.0171
Epoch 74/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 908ms/step - loss: 0.0160
Epoch 74: val_loss improved from 0.01714 to 0.01707, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0160 - val_loss: 0.0171
Epoch 75/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 843ms/step - loss: 0.0160
Epoch 75: val_loss improved from 0.01707 to 0.01696, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0159 - val_loss: 0.0170
Epoch 76/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0156
Epoch 76: val_loss improved from 0.01696 to 0.01683, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0157 - val_loss: 0.0168
Epoch 77/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0156
Epoch 77: val_loss improved from 0.01683 to 0.01665, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0157 - val_loss: 0.0167
Epoch 78/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0155
Epoch 78: val_loss improved from 0.01665 to 0.01655, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0155 - val_loss: 0.0165
Epoch 79/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 310ms/step - loss: 0.0158
Epoch 79: val_loss improved from 0.01655 to 0.01648, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0154 - val_loss: 0.0165
```

```
Epoch 80/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0155
Epoch 80: val_loss improved from 0.01648 to 0.01635, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0153 - val_loss: 0.0164
Epoch 81/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 934ms/step - loss: 0.0153
Epoch 81: val_loss improved from 0.01635 to 0.01627, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0152 - val_loss: 0.0163
Epoch 82/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0154
Epoch 82: val_loss improved from 0.01627 to 0.01617, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0151 - val_loss: 0.0162
Epoch 83/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0152
Epoch 83: val_loss improved from 0.01617 to 0.01608, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0150 - val_loss: 0.0161
Epoch 84/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0148
Epoch 84: val_loss improved from 0.01608 to 0.01601, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0149 - val_loss: 0.0160
Epoch 85/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0146
Epoch 85: val_loss improved from 0.01601 to 0.01593, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0148 - val_loss: 0.0159
Epoch 86/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0147
Epoch 86: val_loss improved from 0.01593 to 0.01582, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0147 - val_loss: 0.0158
Epoch 87/150
1/2 ━━━━━━━━━━──────────  1s 1s/step - loss: 0.0149
Epoch 87: val_loss improved from 0.01582 to 0.01576, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 671ms/step - loss: 0.0146 - val_loss: 0.0158
Epoch 88/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 848ms/step - loss: 0.0144
Epoch 88: val_loss improved from 0.01576 to 0.01569, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0146 - val_loss: 0.0157
Epoch 89/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 845ms/step - loss: 0.0145
Epoch 89: val_loss improved from 0.01569 to 0.01558, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0145 - val_loss: 0.0156
Epoch 90/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 943ms/step - loss: 0.0144
Epoch 90: val_loss improved from 0.01558 to 0.01552, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0144 - val_loss: 0.0155
Epoch 91/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 893ms/step - loss: 0.0142
Epoch 91: val_loss improved from 0.01552 to 0.01549, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0143 - val_loss: 0.0155
Epoch 92/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 879ms/step - loss: 0.0142
Epoch 92: val_loss did not improve from 0.01549
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0143 - val_loss: 0.0156
```

```
Epoch 93/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 908ms/step - loss: 0.0143
Epoch 93: val_loss did not improve from 0.01549
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0144 - val_loss: 0.0163
Epoch 94/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 963ms/step - loss: 0.0150
Epoch 94: val_loss did not improve from 0.01549
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0151 - val_loss: 0.0166
Epoch 95/150
1/2 ━━━━━━━━━━──────────  2s 2s/step - loss: 0.0152
Epoch 95: val_loss did not improve from 0.01549
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s -142316us/step - loss: 0.0148 - val_loss: 0.0156
Epoch 96/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0149
Epoch 96: val_loss improved from 0.01549 to 0.01533, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0147 - val_loss: 0.0153
Epoch 97/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 834ms/step - loss: 0.0141
Epoch 97: val_loss did not improve from 0.01533
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0142 - val_loss: 0.0162
Epoch 98/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 877ms/step - loss: 0.0147
Epoch 98: val_loss did not improve from 0.01533
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0146 - val_loss: 0.0154
Epoch 99/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 911ms/step - loss: 0.0143
Epoch 99: val_loss improved from 0.01533 to 0.01511, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0143 - val_loss: 0.0151
Epoch 100/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0137
Epoch 100: val_loss did not improve from 0.01511
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0140 - val_loss: 0.0156
Epoch 101/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 992ms/step - loss: 0.0142
Epoch 101: val_loss did not improve from 0.01511
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0140 - val_loss: 0.0156
Epoch 102/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0143
Epoch 102: val_loss improved from 0.01511 to 0.01493, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0142 - val_loss: 0.0149
Epoch 103/150
1/2 ━━━━━━━━━━──────────  1s 1s/step - loss: 0.0140
Epoch 103: val_loss did not improve from 0.01493
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 273ms/step - loss: 0.0140 - val_loss: 0.0154
Epoch 104/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 810ms/step - loss: 0.0140
Epoch 104: val_loss did not improve from 0.01493
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0139 - val_loss: 0.0154
Epoch 105/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0138
Epoch 105: val_loss improved from 0.01493 to 0.01485, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0139 - val_loss: 0.0148
Epoch 106/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0136
Epoch 106: val_loss improved from 0.01485 to 0.01483, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0138 - val_loss: 0.0148
Epoch 107/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0135
Epoch 107: val_loss did not improve from 0.01483
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0135 - val_loss: 0.0151
Epoch 108/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0136
Epoch 108: val_loss improved from 0.01483 to 0.01475, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0136 - val_loss: 0.0147
Epoch 109/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0132
Epoch 109: val_loss improved from 0.01475 to 0.01447, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0134 - val_loss: 0.0145
Epoch 110/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - loss: 0.0133
Epoch 110: val_loss did not improve from 0.01447
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0132 - val_loss: 0.0145
Epoch 111/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 355ms/step - loss: 0.0131
Epoch 111: val_loss did not improve from 0.01447
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 942ms/step - loss: 0.0131 - val_loss: 0.0145
Epoch 112/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0131
Epoch 112: val_loss improved from 0.01447 to 0.01429, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0130 - val_loss: 0.0143
Epoch 113/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 829ms/step - loss: 0.0127
Epoch 113: val_loss improved from 0.01429 to 0.01410, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0129 - val_loss: 0.0141
Epoch 114/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0129
Epoch 114: val_loss improved from 0.01410 to 0.01405, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0127 - val_loss: 0.0141
Epoch 115/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0125
Epoch 115: val_loss improved from 0.01405 to 0.01383, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0126 - val_loss: 0.0138
Epoch 116/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 942ms/step - loss: 0.0123
Epoch 116: val_loss improved from 0.01383 to 0.01366, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0124 - val_loss: 0.0137
Epoch 117/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0124
Epoch 117: val_loss improved from 0.01366 to 0.01357, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0123 - val_loss: 0.0136
Epoch 118/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 986ms/step - loss: 0.0122
Epoch 118: val_loss improved from 0.01357 to 0.01339, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s -81156us/step - loss: 0.0122 - val_loss: 0.0134
Epoch 119/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 918ms/step - loss: 0.0120
Epoch 119: val_loss improved from 0.01339 to 0.01329, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0121 - val_loss: 0.0133
Epoch 120/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 980ms/step - loss: 0.0118
Epoch 120: val_loss improved from 0.01329 to 0.01314, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0119 - val_loss: 0.0131
```

```
Epoch 121/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0121
Epoch 121: val_loss improved from 0.01314 to 0.01310, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0117 - val_loss: 0.0131
Epoch 122/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 974ms/step - loss: 0.0116
Epoch 122: val_loss did not improve from 0.01310
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0117 - val_loss: 0.0132
Epoch 123/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0121
Epoch 123: val_loss improved from 0.01310 to 0.01289, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0117 - val_loss: 0.0129
Epoch 124/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0116
Epoch 124: val_loss improved from 0.01289 to 0.01277, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0115 - val_loss: 0.0128
Epoch 125/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0116
Epoch 125: val_loss did not improve from 0.01277
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0115 - val_loss: 0.0136
Epoch 126/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - loss: 0.0121
Epoch 126: val_loss did not improve from 0.01277
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step - loss: 0.0120 - val_loss: 0.0132
Epoch 127/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0119
Epoch 127: val_loss did not improve from 0.01277
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0117 - val_loss: 0.0134
Epoch 128/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0119
Epoch 128: val_loss did not improve from 0.01277
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0118 - val_loss: 0.0128
Epoch 129/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0112
Epoch 129: val_loss improved from 0.01277 to 0.01251, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0113 - val_loss: 0.0125
Epoch 130/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0113
Epoch 130: val_loss improved from 0.01251 to 0.01241, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0111 - val_loss: 0.0124
Epoch 131/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0109
Epoch 131: val_loss did not improve from 0.01241
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0110 - val_loss: 0.0124
Epoch 132/150
1/2 ━━━━━━━━━━──────────  3s 4s/step - loss: 0.0110
Epoch 132: val_loss improved from 0.01241 to 0.01224, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 470ms/step - loss: 0.0109 - val_loss: 0.0122
Epoch 133/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - loss: 0.0109
Epoch 133: val_loss did not improve from 0.01224
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0107 - val_loss: 0.0123
Epoch 134/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0107
Epoch 134: val_loss improved from 0.01224 to 0.01213, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0107 - val_loss: 0.0121
Epoch 135/150
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0106
Epoch 135: val_loss did not improve from 0.01213
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0106 - val_loss: 0.0122
Epoch 136/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0106
Epoch 136: val_loss improved from 0.01213 to 0.01197, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0106 - val_loss: 0.0120
Epoch 137/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0105
Epoch 137: val_loss did not improve from 0.01197
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0104 - val_loss: 0.0120
Epoch 138/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 914ms/step - loss: 0.0104
Epoch 138: val_loss improved from 0.01197 to 0.01184, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0104 - val_loss: 0.0118
Epoch 139/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 951ms/step - loss: 0.0103
Epoch 139: val_loss improved from 0.01184 to 0.01183, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0103 - val_loss: 0.0118
Epoch 140/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0102
Epoch 140: val_loss improved from 0.01183 to 0.01175, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0102 - val_loss: 0.0118
Epoch 141/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 992ms/step - loss: 0.0102
Epoch 141: val_loss improved from 0.01175 to 0.01168, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0101 - val_loss: 0.0117
Epoch 142/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0100
Epoch 142: val_loss improved from 0.01168 to 0.01164, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0101 - val_loss: 0.0116
Epoch 143/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0101
Epoch 143: val_loss improved from 0.01164 to 0.01160, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0100 - val_loss: 0.0116
Epoch 144/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0100
Epoch 144: val_loss improved from 0.01160 to 0.01156, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0100 - val_loss: 0.0116
Epoch 145/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0101
Epoch 145: val_loss improved from 0.01156 to 0.01152, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0099 - val_loss: 0.0115
Epoch 146/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 371ms/step - loss: 0.0100
Epoch 146: val_loss did not improve from 0.01152
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 979ms/step - loss: 0.0099 - val_loss: 0.0116
Epoch 147/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0099
Epoch 147: val_loss did not improve from 0.01152
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0099 - val_loss: 0.0116
Epoch 148/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0098
Epoch 148: val_loss improved from 0.01152 to 0.01141, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0098 - val_loss: 0.0114
Epoch 149/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0098
Epoch 149: val_loss did not improve from 0.01141
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0097 - val_loss: 0.0116
Epoch 150/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 957ms/step - loss: 0.0100
Epoch 150: val_loss did not improve from 0.01141
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0099 - val_loss: 0.0117
Restoring model weights from the end of the best epoch: 148.
```

In [32]:

```python
def show_images(autoencoder, images=None):
    """
    Fonction pour afficher les images originales et leurs
    reconstructions générées par l'autoencodeur.

    Si aucun lot d'images n'est fourni, la fonction récupère un lot
    d'images depuis le générateur de données.

    Arguments :
    - autoencoder : L'autoencodeur entraîné, utilisé pour générer des
    reconstructions des images.
    - images (optionnel) : Un tableau d'images à afficher. Si non spécifié,
    un lot d'images est récupéré du générateur de données.

    Sortie :
    - Affichage des images originales et leurs reconstructions générées par
    l'autoencodeur.
    """

    if images is None:
        example_batch = next(data_flow)
        example_batch = example_batch[0]
        images = example_batch[:10]

    n_to_show = images.shape[0]

    # Génération des reconstructions des images
    reconst_images = autoencoder.predict(images)

    # Création de la figure pour l'affichage
    fig = plt.figure(figsize=(15, 3))
    fig.subplots_adjust(hspace=0.4, wspace=0.4)

    # Affichage des images originales
    for i in range(n_to_show):
        img = images[i].squeeze()
        sub = fig.add_subplot(2, n_to_show, i+1)
        sub.axis('off')
        sub.imshow(img)

    # Affichage des images reconstruites
    for i in range(n_to_show):
        img = reconst_images[i].squeeze()
        sub = fig.add_subplot(2, n_to_show, i+n_to_show+1)
        sub.axis('off')
        sub.imshow(img)
```

Visualisation des images reconstruits aprés avoir entrainé le modéle

In [35]:

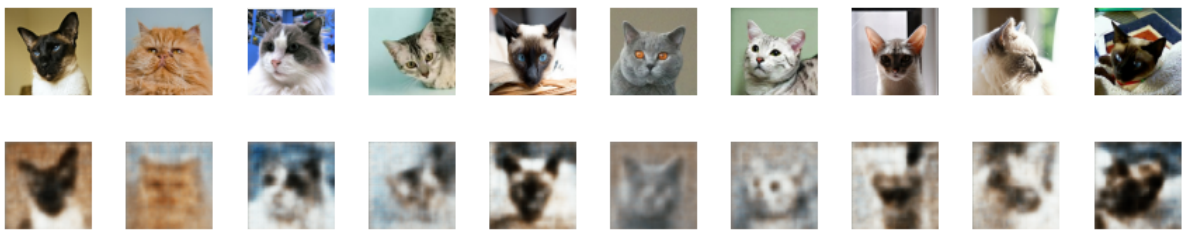
```python
# Les deux fonctions suivantes ne nécessitent pas forcément d'être redéfinies
# Il s'agit plus d'une illustration de ce qu'il faut faire en cas de
# chargement indépendant
```

```
# Fonction de sampling utilisée dans la couche Lambda
@tf.keras.utils.register_keras_serializable()
def sampling(args):
    mean_mu, log_var = args
    epsilon = K.random_normal(shape=K.shape(mean_mu), mean=0., stddev=1.)
    return mean_mu + K.exp(log_var / 2) * epsilon

# Couche personnalisée pour le calcul de la perte KL
@tf.keras.utils.register_keras_serializable()
class KLLossLayer(tf.keras.layers.Layer):
    def call(self, inputs):
        mean_mu, log_var = inputs
        kl_loss = -0.5 * tf.reduce_sum(1 + log_var - tf.square(mean_mu) - tf.exp(l
        self.add_loss(kl_loss)
        return kl_loss

# Le modèle a besoin de connaître la définition de la total_loss
# Compilation du modèle avec la reconstruction loss (la KL est intégrée
# dans l'encodeur)
B = 1.0  # facteur de pondération de la reconstruction

@tf.keras.utils.register_keras_serializable()
def r_loss(y_true, y_pred):
    return K.mean(K.square(y_true - y_pred), axis=[1, 2, 3]) # MSE pixel à pixel

@tf.keras.utils.register_keras_serializable()
def total_loss(y_true, y_pred):
    return B * r_loss(y_true, y_pred) # KL-divergence ajoutée via .add_loss()
```

In [36]:

```
# Chemin du modèle sauvegardé
model_name = "catGeneratingVAE.keras"
model_path = os.path.join(model_dir, model_name)

# Recharger le modèle avec les objets personnalisés nécessaires
vae = tf.keras.models.load_model(
    model_path,
    custom_objects={
        "KLLossLayer": KLLossLayer,
        "sampling": sampling,
        "r_loss": r_loss,
        "total_loss": total_loss
    }
)


show_images(vae)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 183ms/step
```



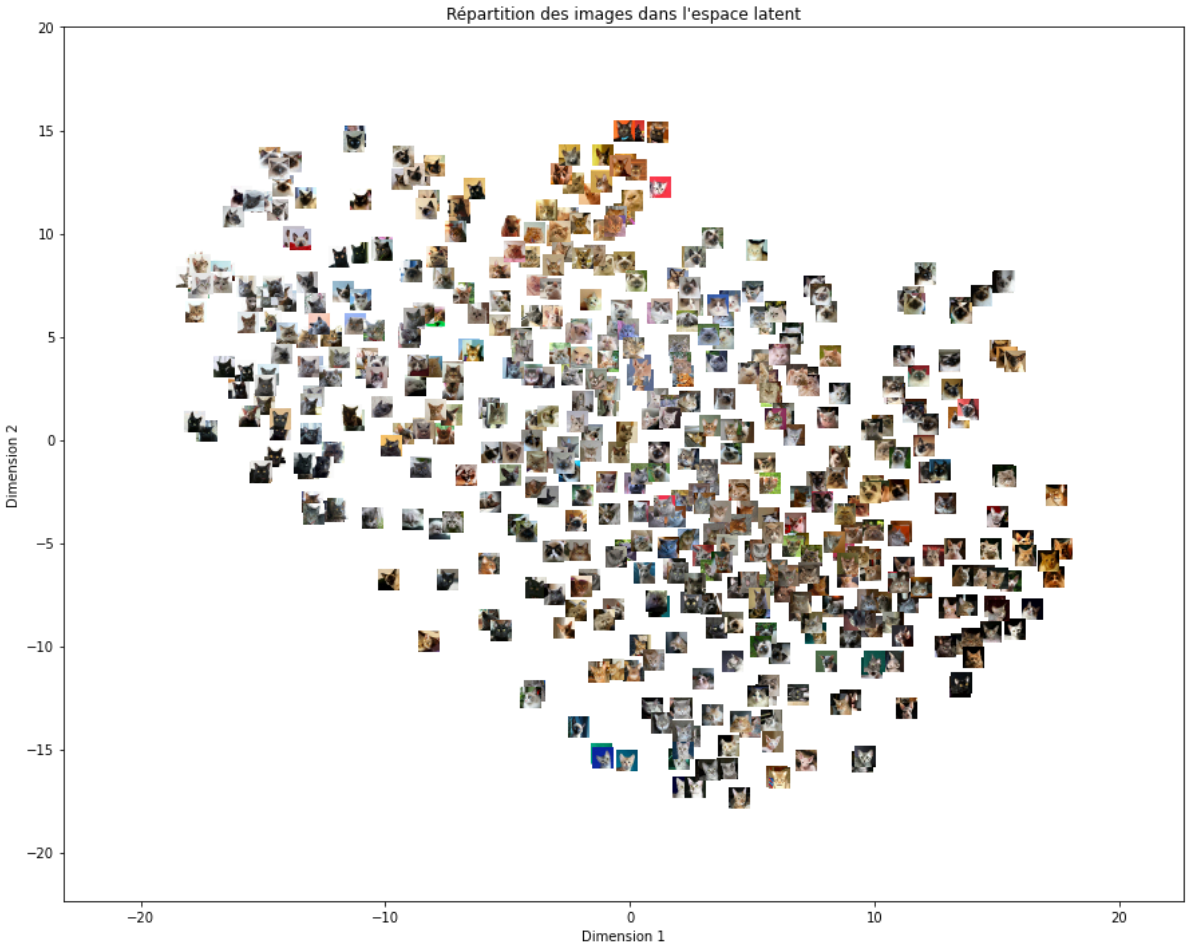
Visualisation de l'espace latent VAE

In [38]:

```
# Chargement du variational autoencodeur
# Chemin du modèle sauvegardé
model_name = "catGeneratingVAE.keras"
```

```python
# Chargement du modèle VAE avec les objets personnalisés
vae_model_path = os.path.join(model_dir, model_name)
vae = load_model(
    vae_model_path,
    custom_objects={
        "KLLossLayer": KLLossLayer,
        "sampling": sampling,
        "r_loss": r_loss,
        "total_loss": total_loss
    }
)

# Visualisation de l'espace latent du VAE
visualize_latent_space(
    encoder_model=vae_encoder,
    images=example_batch,
    num_images=batch_size,
    image_size=(32, 32),
    zoom=0.5
)
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
```

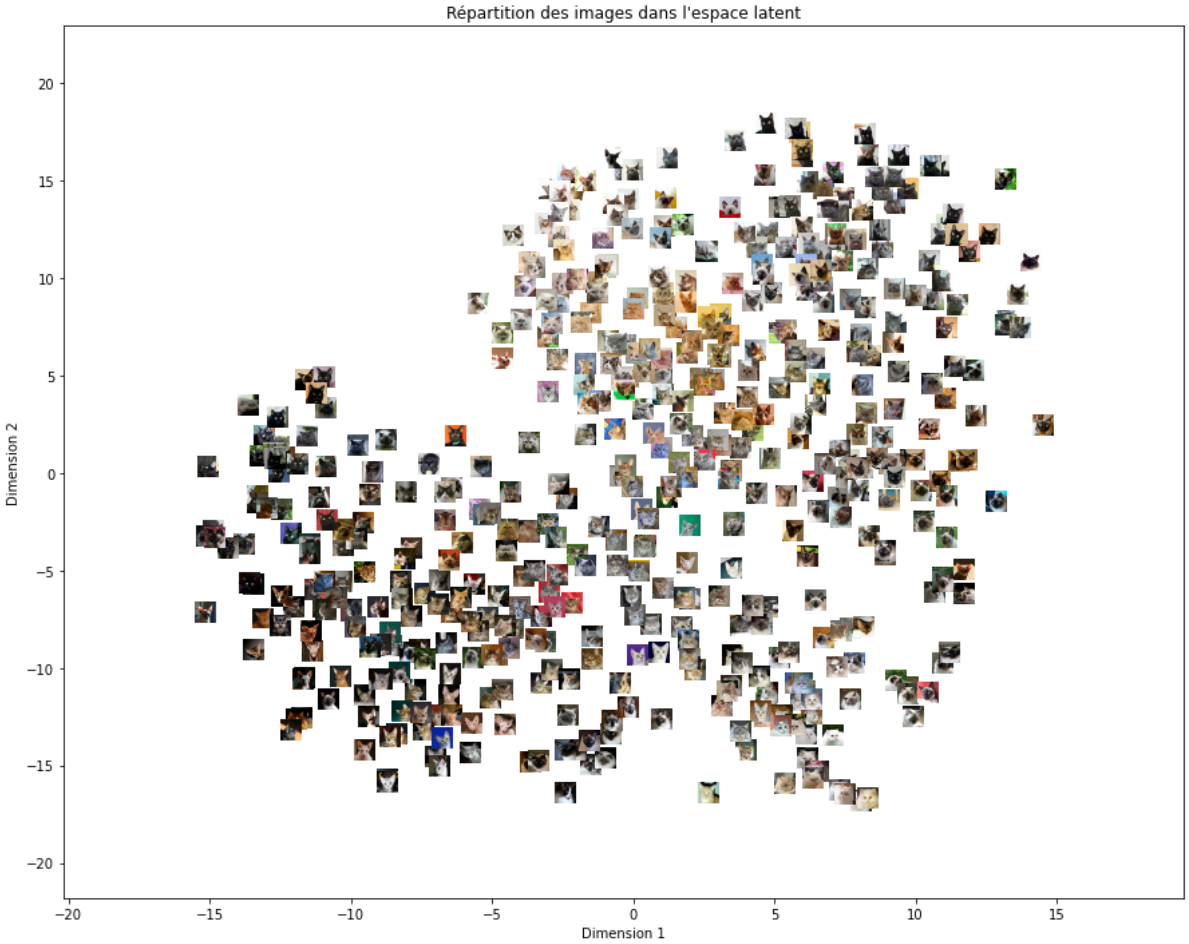


In [39]:

```python
#comparaison avec autoencodeur sans VAE:
# Chemin vers le modèle entraîné
model_path = os.path.join(model_dir, "catmodeltrain.keras")

# Chargement du modèle
autoencoder = load_model(model_path)

# Récupération du premier lot d'images
example_batch = next(data_flow)

# Extraction des images
```

```
example_batch = example_batch[0]
# Visualisation de l'espace latent avec les vignettes
visualize_latent_space(
    encoder_model=encoder,
    images=example_batch,
    num_images=batch_size,
    image_size=(32, 32),
    zoom=0.5
)
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step
```



In [40]:

```
##idk if theres much difference???
#celle de VAE semble un peu plus compacte?
```

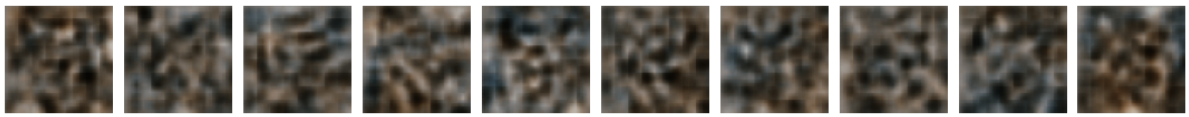
Génération des images:

In [41]:

```
# Chemin du modèle sauvegardé
model_name = "catGeneratingVAE.keras"
# Chargement du modèle VAE avec les objets personnalisés
vae_model_path = os.path.join(model_dir, model_name)
vae = load_model(
    vae_model_path,
    custom_objects={
        "KLLossLayer": KLLossLayer,
        "sampling": sampling,
        "r_loss": r_loss,
        "total_loss": total_loss
    }
)

_ = vae.predict(example_batch)  # passe des images pour activer le modèle
```

```
# Nombre d'images à générer
num_images = 10

# Tirage aléatoire de vecteurs latents depuis une normale centrée réduite
latent_vectors = np.random.normal(loc=0.0, scale=1, size=(num_images,
                                                          latent_dim))

vae_decoder = vae.get_layer('decoder')

# Génération des images à partir du décodeur du VAE
generated_images = vae_decoder.predict(latent_vectors)

# Affichage des images générées
plt.figure(figsize=(15, 3))
for i in range(num_images):
    ax = plt.subplot(1, num_images, i + 1)
    plt.imshow(generated_images[i])
    plt.axis("off")

plt.suptitle("Images générées aléatoirement à partir de l'espace latent (VAE)")
plt.tight_layout()
plt.show()
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 222ms/step
```



In [42]:
```
#c'est normal d'avoir ce résultat vu que le tirage des points était aléatoire
```

Ici on fait échantillonner les vecteurs latents qui se trouvent dans les zones denses pour produire un résultat plus réaliste

In [43]:
```
# Sous-modèle pour obtenir les μ
encoder_mu_model = Model(inputs=vae.input,
                         outputs=vae.get_layer('mu').output)

# Obtenir les μ
mus = encoder_mu_model.predict(example_batch)

# Tirage aléatoire autour des μ
epsilon = np.random.normal(size=mus.shape)
scale = 1
latent_samples = mus + scale * epsilon

# Génération d'images à partir du décodeur
vae_decoder = vae.get_layer('decoder')
generated_images = vae_decoder.predict(latent_samples)

# Affichage
plt.figure(figsize=(20, 4))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(generated_images[i])
    plt.axis("off")
plt.suptitle("Échantillons latents proches des vrais μ")
plt.show()
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 42ms/step
```

Échantillons latents proches des vrais μ

In [ ]:

```
#deja des résultats bcp mieux!
```